M4010104 / M4020104: PROGRAMMING IN PYTHON · DIGITAL UNIVERSITY KERALA · CAPSTONE PROJECT SPECIFICATION

Project 21: Academic Performance Early-Warning System & Study-Habit Correlator

Effort Budget: 10–12 Hours | Core Stack: Pure Python, NumPy, Pandas, Matplotlib, Seaborn, Plotly | Max Marks: 25 | Instructor: Dr. Sunil TT

1. Executive Overview & Domain Context

Educational data mining models behavioral and academic attributes—including weekly self-study hours, class attendance percentages, prior exam marks, tutoring sessions, and extracurricular participation—to forecast final course outcomes. Designing a transparent early-warning predictor enables academic faculty to provide targeted remedial mentoring before terminal exams.

Your team will develop an academic analytics and early-warning engine in pure Python that cleans student academic records, checks grade bounds [0, 100], fits a multivariable ordinary least-squares regression model in NumPy, identifies critical study-hour failure thresholds, and delivers interactive visualizers answering the 12 core questions below.

Primary Public Dataset & Telemetry/Data Schema

The project utilizes the verified public Student Performance Prediction Dataset on Kaggle (**`https://www.kaggle.com/datasets/amrmaree/student-performance-prediction`**). The dataset structure captures:

- **Raw Feature Attributes:** **`student_id`**, **`study_hours_per_week`**, **`attendance_percentage`**, **`previous_exam_score`**, **`tutoring_sessions`**, **`extracurricular_participation`**, **`final_exam_score`**.
- **Engineered / Target Variables:** **`study_attendance_product`**, **`predicted_final_score`**, **`performance_tier`** (Distinction, Pass, At-Risk), **`score_residual`**.

2. Complete Project Directory & File Architecture

Each team must organize their repository into a standardized, production-grade layout. All application source code must reside inside a dedicated package (**`src/`**), with separate directories for raw/curated data, interactive development notebooks, visual figures, and formal documentation:

```
academic-early-warning-system/
├── README.md                          # Repository documentation & GitHub Pages source
├── requirements.txt                   # Dependency specification (numpy, pandas, matplotlib, seaborn, p
├── run_pipeline.py                    # Standalone production CLI application entry point
├── data/
│   ├── raw/
│   │   └── student_performance_raw.csv            # Raw uncleaned input dataset (including simulated/in
│   └── processed/
│       ├── clean_student_analytics.csv          # Curated, validated clean CSV dataset
│       └── student_analytics_audit.log           # Timestamped audit report detailing filtered/rejected
├── src/
│   ├── __init__.py                    # Python package initialization
│   ├── ingestion.py                   # Defensive line-by-line file reader, range auditor & CSV exporte
│   ├── analyzer.py                    # AcademicEarlyWarningSystem OOP class, NumPy vectorization & Pan
│   └── visualizer.py                  # Matplotlib envelope plots, Seaborn heatmaps & Plotly HTML expor
├── notebooks/
│   └── development_cycle.ipynb        # Development cycle notebook: EDA, trials, tests & unit assertion
└── reports/
    ├── figures/
    │   ├── primary_envelope.png       # Matplotlib confidence envelope & regression overlay
    │   ├── correlation_heatmap.png    # Seaborn annotated feature correlation heatmap
    │   └── distribution_regimes.png   # Seaborn distribution across categorical/domain tiers
```

```
    ├── interactive_dashboard.html     # Standalone interactive Plotly HTML dashboard with controls
    └── Project_Report.pdf             # 3-4 page final technical report
```

3. Dual Development Deliverables: Notebook vs. Standalone Program

To mirror industry software engineering standards, each student group must deliver two distinct, complementary implementations of their solution:

Deliverable A: The Development Cycle Notebook (`notebooks/development_cycle.ipynb`)

The development notebook chronicles your team's engineering journey from raw data exploration to validated model prototypes. It is not merely a final script pasted into cells, but an authentic record of the development cycle:

- **Phase 1: Exploratory Data Ingestion & Diagnostics:** Loading raw data, inspecting missing values, testing boundary rules, and logging rejected records.
- **Phase 2: Mathematical Formulation & Feature Verification:** Developing and validating NumPy array calculations against target benchmarks using **`np.allclose()`**.
- **Phase 3: OOP Class Architecture & Unit Tests:** Instantiating the **`AcademicEarlyWarningSystem`** class, executing automated unit assertions (e.g., verifying that passing invalid parameters raises an explicit **`ValueError`**), and testing edge cases.
- **Phase 4: Statistical Modeling & Regimen Exploration:** Implementing ordinary least-squares regression, rolling-window calculations, and multi-metric Pandas grouping.
- **Phase 5: Visual Prototype Development:** Iteratively designing and refining Matplotlib and Seaborn figures, followed by configuring the interactive Plotly HTML dashboard.

Deliverable B: The Final Standalone Python Program (`run_pipeline.py` & `src/`)

While the notebook documents the exploratory development cycle, your repository must also provide a production-ready, fully executable Python application. The program must run end-to-end directly from the terminal without requiring Jupyter:

- **Command-Line Interface (CLI):** Executed via **`python run_pipeline.py --input data/raw/student_performance_raw.csv --output-dir data/processed --report-dir reports`**.
- **Modular Package Import:** The driver script imports cleanly separated domain logic from **`src.ingestion`**, **`src.analyzer`**, and **`src.visualizer`**.
- **Terminal Feedback & Exit Codes:** The program must print structured progress indicators, log clean/faulty record counts to the console, and return an exit code of **`0`** upon successful pipeline completion.
- **Automated Artifact Generation:** In a single execution pass, the script reads raw inputs, writes **`data/processed/clean_student_analytics.csv`**, saves **`data/processed/student_analytics_audit.log`**, exports static figures to **`reports/figures/`**, and generates the standalone **`reports/interactive_dashboard.html`**.

4. Technical Investigation Requirements & Research Questions (Q1–Q12)

Your team's implementation across the development notebook and standalone program must rigorously answer the following 12 course-aligned engineering questions:

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
File I/O & Exceptions (**`src/ingestion.py`**)	**Q1**	**Academic Score & Percentage Bounds Audit:** Parse raw student CSV lines using try-except. Validate attendance percentage in [0, 100], exam marks in [0, 100], study hours in [0, 60], and log corrupt entries.	open(), try-except (ValueError, FileNotFoundError), range checks

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
Data Curation & Export (`src/ingestion.py`)	**Q2**	**Curated Student Academic Export & Audit Log:** Write clean records to clean_student_analytics.csv while logging duplicate student IDs, negative scores, and non-numeric entries in student_analytics_audit.log.	Defensive file writing, dictionary parsing, formatted log reporting
OOP Design (`src/ingestion.py`)	**Q3**	**Academic Early-Warning System Class:** Design an encapsulated class AcademicEarlyWarningSystem with methods load_from_csv(), predict_at_risk_students(pass_mark=50.0), and summary_report(). Ensure pass_mark < 0 raises ValueError.	OOP class, __init__, encapsulation, raise ValueError
NumPy Arrays (`src/analyzer.py`)	**Q4**	**Vectorized Feature Interaction & Verification:** Using only raw NumPy arrays of study hours and attendance, compute the interaction cross-product (Study_Hours * Attendance) and min-max scaled indices. Verify via np.allclose().	np.multiply(), np.min(), np.max(), np.allclose()
NumPy Masking (`src/analyzer.py`)	**Q5**	**At-Risk Failure vs. Distinction Cohort Masking:** Flag At-Risk students (Final_Exam < 50) in NumPy. Determine the study-hours and attendance thresholds below which course failure probability exceeds 50% using boolean array masks.	np.mean(), np.std(), boolean masks (&, |), Python set
NumPy Regression (`src/analyzer.py`)	**Q6**	**Multivariable Exam Score OLS Regression Model:** Fit an ordinary least-squares model (Final_Score = b0 + b1*Study_Hours + b2*Attendance + b3*Prior_Score) via np.linalg.lstsq. Quantify relative predictive weights of attendance vs. study hours.	np.column_stack(), np.linalg.lstsq(), matrix product (@), R2
Pandas Series (`src/analyzer.py`)	**Q7**	**Cohort Score Distribution & Decile Binning Audit:** Compute decile bins of prior exam scores in Pandas. Audit the correlation between weekly study hours and final score across tutoring vs. non-tutoring cohorts.	.groupby('tutoring_sessions').agg(), .describe()
Pandas Groupby (`src/analyzer.py`)	**Q8**	**Attendance-Study Hours Discretization & Pivot Table:** Discretize Attendance into Low (< 75%), Medium (75-85%), High (> 85%); discretize Study_Hours into Low, Medium, High. Build a 3×3 pivot_table(margins=True) showing failure rates.	pd.cut(), .groupby().agg(), .pivot_table(margins=True)
Pandas Reshaping (`src/analyzer.py`)	**Q9**	**Wide-to-Long Academic Attribute Ranking via Melt:** Reshape behavioral metrics (Study_Hours, Attendance, Prior_Score) into tidy long format via pd.melt() and extract top 5 students showing highest score improvement.	pd.melt(), .groupby('Metric'), .nlargest(5)
Matplotlib (`src/visualizer.py`)	**Q10**	**Matplotlib Actual vs. Predicted Exam Score Plot:** Plot Actual vs. Predicted Final Exam Score with Matplotlib. Overlay the ideal y = x identity line, failure boundary threshold (y = 50), and custom legends.	plt.subplots(), ax.scatter(), ax.plot(), ax.axhline()

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
Seaborn (`src/visualizer.py`)	**Q11**	**Seaborn Attendance vs. Study Hours Jointplot & Heatmap:** Render an annotated correlation heatmap across academic metrics with sns.heatmap(). Create an attendance vs. study-hours jointplot colored by performance tier with sns.jointplot().	df.corr(), sns.heatmap(), sns.jointplot(), sns.despine()
Plotly HTML (`src/visualizer.py`)	**Q12**	**Interactive Academic Early-Warning Dashboard:** Build an interactive HTML dashboard (interactive_dashboard.html) displaying student performance with grade cutoff sliders, risk filters, and cohort selectors.	plotly.express.scatter(), custom hover tooltips, write_html()

5. Collaborative Git Workflow, GitHub Pages & README.md Specification

Best Practices: Collaborative Git Workflow & GitHub Pages Deployment from Day 1

From the very first day, the student team must adopt professional version control and documentation practices. On Day 1, one designated team member initializes the central public GitHub repository, establishes a clear branch protection rule preventing direct pushes to **`main`**, adds all teammates as registered **Contributors** (with write access), and grants read access to the Course Instructor (**`Dr. Sunil TT`**) and teaching assistants. Immediately thereafter, the team must activate **GitHub Pages** (via *Settings → Pages → Build and deployment: Deploy from a branch: main /docs or root*) so that project documentation, interactive architecture diagrams, and the live Plotly dashboard (**`reports/interactive_dashboard.html`**) are publicly accessible and continuously deployed throughout the development lifecycle. All feature development must follow a strict branching model (e.g., **`feature/ingestion`**, **`feature/analysis`**, **`feature/visualization`**): individual team members must write small, atomic, well-commented commits from their own personal GitHub accounts and merge them into **`main`** exclusively through formal **Pull Requests (PRs)** accompanied by peer review. Dumping entire codebases in a single, monolithic end-of-semester commit is strictly prohibited and will result in zero marks for individual Git contribution; maintaining an unbroken, transparent Git commit and PR audit trail is essential for passing the code ownership defense during the live viva-voce examination.

Repository README.md Specification Template

The root **`README.md`** serves as the primary technical landing page and must strictly conform to the following structural format:

````
# Project 21: Academic Performance Early-Warning System & Study-Habit Correlator
[![Course](https://img.shields.io/badge/Course-Programming%20in%20Python-blue.svg)](https://duk.ac.in)
[![Python](https://img.shields.io/badge/Python-3.11%2B-brightgreen.svg)](https://python.org)
[![License](https://img.shields.io/badge/License-MIT-yellow.svg)](LICENSE)

## 1. Project Overview & Domain Context
- **Institution:** Digital University Kerala (MTech AI / MSc AI)
- **Course:** M4010104 / M4020104 – Programming in Python | **Instructor:** Dr. Sunil TT
- **Dataset:** Student Performance Prediction Dataset on Kaggle (https://www.kaggle.com/datasets/amrmare

## 2. System Architecture & Pipeline Design
[Provide an ASCII or Mermaid architecture flow diagram showing raw ingestion -> validation -> analyzer -

## 3. Installation & Environment Setup
```bash
git clone https://github.com/<your-team>/academic-early-warning-system.git
cd academic-early-warning-system
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```
````

````
## 4. Execution Instructions
### A. Running the Standalone Application
```bash
python run_pipeline.py --input data/raw/student_performance_raw.csv --output-dir data/processed --report
```
### B. Running the Development Notebook
```bash
jupyter notebook notebooks/development_cycle.ipynb
```

## 5. Key Analytical & Statistical Findings (Q1-Q12 Summary)
[Provide a concise markdown summary table detailing raw vs. clean counts, regression coefficients, anoma

## 6. Interactive Telemetry Dashboard
Explore the live interactive Plotly visualization hosted on GitHub Pages:
🔗 [Live Project Dashboard](https://<your-team>.github.io/academic-early-warning-system/interactive_dash

## 7. Contributor Git Commit & Pull Request Log Table
| Contributor Name | GitHub Handle | Assigned Module / Tasks | Verified Commit Hash Range | Merged Pull
| :--- | :--- | :--- | :--- | :--- |
| [Student Full Name] | `@github_handle` | Modular Architecture (`src/ingestion.py`, `src/analyzer.py`,
````

6. Evaluation Rubric & Mandatory Academic Integrity Governance

Projects are evaluated on a 25-mark scale based on code quality, mathematical correctness, visualization depth, git collaboration discipline, and live technical defense:

Assessment Criterion	Basis of Evaluation & Deliverable Verification	Max Marks
1. Data Curation & File I/O Pipeline (Q1–Q2)	Robust line-by-line file reading with defensive **`try-except`** handling, physical/domain bounds validation, clean CSV output, and structured **`student_analytics_audit.log`** reporting.	**5 Marks**
2. OOP, NumPy & Pandas Analytics (Q3–Q9)	Modular **`AcademicEarlyWarningSystem`** class architecture, vectorized NumPy feature re-derivations (verified via **`np.allclose`**), regression modeling, and Pandas rolling/groupby/melt operations.	**6 Marks**
3. Multi-Library Visualizations (Q10–Q12)	Publication-quality Matplotlib confidence envelope/trend plot, Seaborn correlation heatmap and distribution violin plot, and standalone interactive Plotly HTML dashboard.	**5 Marks**
4. Software Packaging, GitHub & Documentation	Clean directory layout, comprehensive **`README.md`**, working GitHub Pages site, **`development_cycle.ipynb`** notebook, standalone **`run_pipeline.py`**, and balanced individual Git commits/PRs.	**4 Marks**
5. Live Presentation & Code Viva-Voce Defense	Technical walkthrough and individual live viva defense where each student must explain or modify lines of code in real time without external assistance.	**5 Marks**

> **STRICT ACADEMIC INTEGRITY & ZERO-TOLERANCE AI POLICY (0 MARKS PENALTY):** All submitted Python source code (**`.py`**, **`.ipynb`**) and written documentation will be screened through an automated **AI-Code & Plagiarism Detection Suite**. If AI-generated code or synthetic documentation is detected at any checkpoint or final submission, **0 Marks will be awarded immediately** for the entire project. Each student must prove authentic individual code ownership during the live viva-voce examination — *Course Instructor: Dr. Sunil TT*.

7. Milestone Schedule, Submission Checklist & Viva Defense Prompts

To ensure systematic progress across the semester, teams must adhere to three project checkpoints and prepare for on-the-spot live code modification challenges during the viva-voce examination:

Milestone Stage	Scheduled Window	Required Deliverables & Verification Milestones
Checkpoint 1 (Data Ingestion)	Week 11 Lab Session	Repository initialized with contributors and GitHub Pages active. **`src/ingestion.py`** operational with line-by-line filtering, **`clean_student_analytics.csv`** exported, and **`student_analytics_audit.log`** generated.
Checkpoint 2 (Analytics & OOP)	Week 13 Lab Session	**`src/analyzer.py`** complete with **`AcademicEarlyWarningSystem`** class, NumPy feature checks (Q4), outlier masks (Q5), regression modeling (Q6), and exploratory notebook sections.
Final Submission & Viva	Week 15 Project Review	Full package operational: standalone **`run_pipeline.py`**, polished **`development_cycle.ipynb`**, interactive **`interactive_dashboard.html`** on GitHub Pages, 3–4 page PDF report, and live code defense.

Representative Live Viva-Voce Code Modification Challenge

During the individual code viva-voce, the instructor will ask students to execute an immediate 2-minute code modification on their working repository to demonstrate genuine authorship:

- **Challenge Prompt:** *"In src/analyzer.py, adjust the definition of 'At-Risk' from Final Exam Score < 50 to Final Exam Score < 60. Re-run your module and compute the new weekly study-hours threshold required to achieve a < 10% failure rate."*